

Implementing Risk-Based Testing by Computational Intelligence of Software Quality Metrics



Vinita Malik

Abstract This research has implemented an efficient risk-based testing technique where the prioritization of test cases is done on the basis of ranking of component risks and quality control metrics by automatic source code analysis in evolutionary environments to correlate risk, quality assurance, and testing life cycle. The literature on various risk-based testing techniques has been analyzed critically, and then a novel and original technique has been proposed and implemented to correlate risks, software testing life cycle, and quality assurance. The proposed test case prioritization approach has been compared with no prioritization, random prioritization, and reverse prioritization to prove the proposed work effectiveness. The proposed risk-based testing approach and model quality is assured in terms of reduced test execution time and percentage of test suites executed to cover all the faults.

Keywords Risk management · Risk-based testing · Quality metric

1 Introduction

It is indicated in statistics that 53.3% of IT projects are going behind schedule as well as budget and are not even potent to give the software features as per the required specifications. Around 31% of the projects are terminated untimely and only 62% are able to provide the primary requirement features. The primary cause is the risks inherent in the software projects [1]. Risks are stated as the probability of harm or loss from an engineering project that may have adverse consequences in terms of environment/cost or safety [2]. The software project risks are defined by 02 factors, i.e., failure severity and the probability of event happening [3]. Software project risk management is primarily defined as the set of practices and principles aiming at identification, assessment, and planning on risk factors for improving the possibilities of a successful project outcome [4]. The risks are identified by check listing, brainstorming, interviewing, surveying, and assessed by a decision tree,

V. Malik (✉)

Information Scientist at Central University of Haryana, Adalpur, Haryana, India

e-mail: is@cuh.ac.in

probability, questionnaire, metrics, fuzzy methods, and exposure values [5–7]. Risks are planned by consultants, market research, classification, and cognitive theories, and monitored by surveys and rough set theory [8, 9]. The life cycle of software testing encompasses a testing way which is carried out to meet the software quality goals. It comprises many different activities which get accomplished to achieve a good quality product. The different phases in STLC are mentioned as, i.e., analysis of test requirements, test cases planning, designing test cases, their execution, and test cycle closure [10]. The integrated usage of risk management and software testing has helped in improving the software quality either by using the testing process to assist the risk management process or by risk management to assist the testing process [11]. Risk assessment and testing in combination, i.e., risk-based testing help in targeting the most critical defects by gathering the risk information [12].

This research contributes by proposing and implementing a risk-based testing technique on source code analysis by an intelligent vendor and quality metric to improve overall software quality. It also elaborates literature on risk-based testing techniques employed in the past. This paper also establishes the correlation among testing, risk management and quality assurance.

2 Background

2.1 Testing Based on Software Risks

2.1.1 Motivation

Testing based on risks in software projects begins early by identifying risks to preserve the system quality, and this knowledge of risk may be used to lead the planning of test cases, specification, their preparation, and execution. This type of testing is also utilized to provide opportunities to lessen the likelihood of the defect. It also measures how well defects are found and removed from the critical areas. The knowledge of risks may be utilized to regulate the testing extent and the overall efforts as well as the test budgets. Project risks may be used as the rationale and motivation for quality assurance in various software applications.

2.1.2 Advantages and Disadvantages

The advantages of risk-based testing include critical functions testing, understanding business aspects by test case prioritization, saving project resources, and more focused testing to improve customer satisfaction. But disadvantages include subjective risk assessment overhead, difficulty in choosing right stakeholders and resource overhead [13].

2.1.3 Literature

Risk-based testing helps in informed decision-making by delivering the most optimal test, by selecting better strategies and test objects/cases. It depicts a methodology to not only to prioritize the test cases as per deadlines but the test cases are optimally reduced. The literature for the risk-based testing is given as follows in Table 1.

Table 1 Risk-based testing techniques

S. No	Author name, year	Technique	Model
1	Kloos et al. [14]	Model-based	Test modeling by fault tree analysis and test case prioritization based on severity levels in safety-critical systems
2	Yoon [15]	Risk exposure is used to prioritize test cases, test case correlation, and mutation analysis	–
3	Zech [16]	Model-based	Risk modeling from the vulnerability database. Test execution based on misuse case model for cloud systems
4	Bai et al. [17]	Dependencies in the ontologies considered for probability and impact evaluation in risk- based testing in service-oriented systems and testing cost and efficiency are compared with random testing	–
5	Felderer et al. [18]	Test planning and risk analysis executed in parallel	Factor-criteria-metrics based model
6	Felderer [19]	Integrating risk-based testing is used into the complete testing process	–
7	Ray and Mohapatra [20]	Sequence diagrams and state machine to guide risk analysis in risk-based testing as per complexity and severity	–
8	Ramler and Felderer [21]	Testing activities based on defect prediction in the risk-based testing process	–
9	Grossmann and Felderer [22]	Risk-based taxonomy as an approach to RBT	–
10	Ramler et al. [23]	Probability calculation by extreme programming in RBT	–

3 Design and Implementation of Proposed Work

In this proposed research work, first, five dynamic software projects are downloaded from the Github repository, belonging to different evolutionary environments. The five dynamic projects downloaded from Github repository, i.e.,

PowerReader_Smart (Pervasive software application, Hpg_Bigdata (Big data application), SmartTV_IoT (Internet of Things application), OpenNebula_Cloud (Cloud application), Pybrain_AI (Artificial Intelligent application) [24–28]. The applications support the technologies from the technology set {JavaScript, Ksh, Python, C, C++, Java, C#, Lua, Matlab}. The evolutionary application's source code is scanned by an intelligent vendor to perform the source code analysis [29]. Once the applications are scanned, software quality metrics like resiliency, elegance, agility, impact, and files with the highest risks, are evaluated. Risks indicators and software health factors that are evaluated by the smart vendor include programming practices, code patterns, embedded documentation, code readability, code security, and cyclomatic complexity. The software health factors and risks inherent in code is based on the real applications assessment by smart vendor. More is the risks found in files of the application scanned, less is the software health. The vendor is capable to spot the potential defects for large-scale and evolutionary applications supporting around 40 technologies. The software health is defined by the given formula [30].

$$\text{Software Quality} = (\text{Software Agility} + \text{Software Resilience} + \text{Software Elegance})/3$$

where software quality is defined by how the application components comply with the software code quality that increases resiliency and agility and reduces software complexity. The software health or quality is categorized by the smart vendor in the high, medium, and low classes. Once the source code is scanned for all evolutionary applications, then the software application with the lowest health is considered at the highest priority. Then the components of the application with the lowest health and highest risks are taken as the main case study and the components ranking is done as per calculated ROCR index. ROCR index is derived from software resiliency, elegance, agility, and business impact which are quality metrics. The test cases are planned and created for each component. The component having the highest ROCR index and a high number of risky files will be tested first. The test plan is uploaded in a CSV file as per the format supported by the test management tool, i.e., Qase [31]. Other similar risk-based testing tools.

i.e., may be used for test case execution [32–34]. Test cases get executed once uploaded to repository. The test case execution time gets reduced when the test cases are executed as per the order of prioritization. Also, less percentage of the test suite is required to be executed for the prioritized order assuring overall software quality by reducing the testing effort. ROCR index is used to rank application components based on quality metrics like resiliency (SR), agility (SA), business impact (BI), and elegance (SE). It is a composite metric to calculate the software components risks on the scale of 0–100 with a weighted average formula, given below [30]:

$$\begin{aligned}
 \text{ROCR} = & (((((100 - \text{Software resiliency}) * 7) \\
 & + ((100 - \text{software elegance}) * 2) \\
 & + ((100 - \text{software agility}) * 1)) / 10) \\
 & \times \text{Business impact}) / 100) / \text{No. of components}
 \end{aligned}$$

3.1 *The Proposed Technique*

The proposed risk-based testing technique is described stepwise as below:

- i. First, 05 evolutionary software project source codes are downloaded from the Github repository and scanned with the smart vendor to perform source code analysis.
- ii. The files with the highest risks, software resiliency, agility, elegance, and business impact quality metrics are evaluated automatically by an intelligent vendor, once the source code is scanned. The application with the lowest health is considered for testing at the highest priority.
- iii. The ranking of component risks (ROCR) index is derived from the quality metrics for the main components of the application with the lowest health.
- iv. The test cases are designed component wise as per calculated ROCR index as well as the number of riskiest files, and accordingly, the test cases are prioritized.
- v. The test cases get executed once the test cases repository get uploaded in test management tool in CSV format.
- vi. The proposed system quality is assured in lesser test execution time and test cases executed percentage to cover all of the system faults.

3.2 *Model*

The proposed technique is designed as given below in Fig. 1:

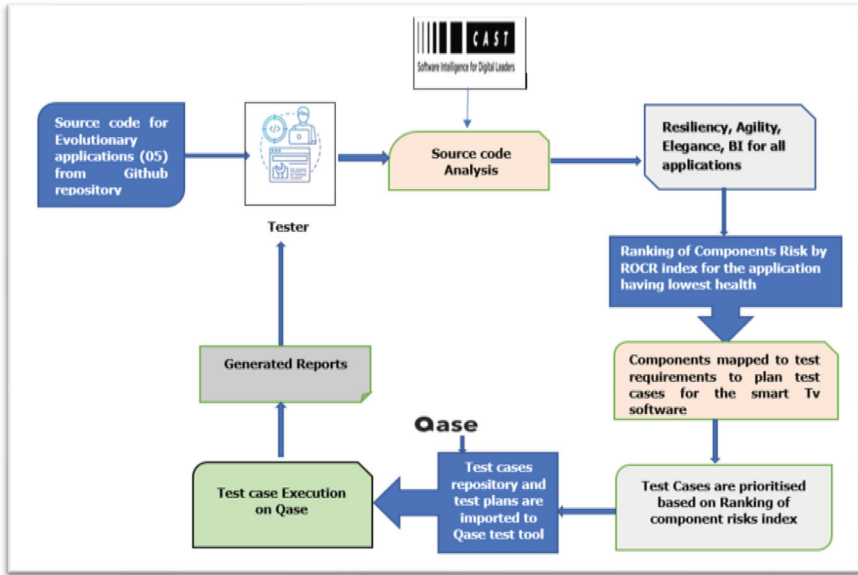


Fig. 1 Proposed model for the proposed technique

3.3 Implementation

In the proposed technique, test cases have been prioritized based on the ranking of components risks index, which has been derived from quality metrics, i.e., software resiliency, agility, elegance, and business impact by smart vendor [29]. Here the ranking of component risk is most impacted by the software resiliency as it targets all software reliability risks and programming errors. The ranking is least impacted by software agility which is meant for documentation risks and elegance gives the risks related to code complexity. That is why the ratio of Software Resiliency (SR): Software Elegance (SE): Software Agility (SA) is taken as 7:2:1 on a scale of 10. In the proposed work, where the application with the lowest health has been tested on the Qase test management tool, which has been utilized to formulate the correlation of risks, software testing life cycle, and system quality. The achievement of the proposed work is estimated by the overall test execution time and % of test suites executed to cover all the faults. The source code scans the evolutionary applications and Smart TV is found to have the lowest software health as given in Table 2.

Then, smart TV is taken as the case study, and based on the component quality metrics, ROCR is calculated as per the proposed formula which has been given in Table 3. The test cases are prioritized as per the ROCR index as given in Table 4.

Table 2 Applications indicating their software health

S. No	Application name	Resiliency	Agility	Elegance	Quality/health
A1	PowerReader_Smart	69.72	63.79	81.12	71.54
A2	Hpg_Bigdata	79.98	59.66	76.34	71.99
A3	Awesome_SmartTv_ _IoT	80.77	71.22	40.94	64.31
A4	OpenNebula_Cloud	87.81	65.56	92.41	81.92
A5	Pybrain_AI	88.89	42.59	100	77.16

Table 3 Component-wise metrics evaluation by the smart vendor

Id	Component name	Resiliency y	Agility	Elegance	Business Impact	#Files with the highest risk	ROCR index
C1	Navigation	73.7	66.1	77.9	47.9	2	2.31
C2	Audio_Control	54.96	55.1	58.67	31.9	4	2.83
C3	Video_Stream	65.44	68.06	74.11	48.48	4	3.15
C4	Network	63.81	46.88	35.35	37.82	5	3.29
C5	Channel_List	77.3	47.3	65.6	38.32	1	2.14

Table 4 Test case prioritization order as per ROCR index

Id	Component name	Requirements	Test cases	ROCR index	Test case prioritization order on ROCR
C1	Navigation	R1, R2	T1, T2	2.31	T7, T8, T5, T6, T3, T4, T1, T2, T9, T10
C2	Audio_Control	R3, R4	T3, T4	2.83	
C3	Video_Stream	R5, R6	T5, T6	3.15	
C4	Network	R7, R8	T7, T8	3.29	
C5	Channel_List	R9, R10	T9, T10	2.14	

The components test case requirements are identified and mapped with test cases. The test plan is created and uploaded as a CSV file to the Qase test management tool and test cases are executed. The test cases are imported in test management tool. The test plan is created for the smart application and test cases are executed in various orders to achieve the desired results.

4 Results

In the proposed RBT technique where test case prioritization is done based on the ROCR index has been compared with the following techniques:

- i. No Prioritization, i.e., the test case order remains untreated, i.e., T1.....T10
- ii. Random Prioritization: The test cases are randomly ordered, i.e., T1, T4, T7, T10, T9, T2, T3, T5, T8, T6.
- iii. Reverse Prioritization: The test cases have been ordered in reverse T10.... T1.
- iv. Proposed order: The test cases are ordered based on the ranking of components risks, i.e., T7, T8, T5, T6, T3, T4, T1, T2, T9, T10.

For system quality assurance, the proposed work has been compared with other techniques in terms of test execution time and percentage of test cases executed to cover the faults. The comparison of the proposed system with other techniques is given as per Figs. 2 and 3 as given below.

The results for % of test cases executed to cover the faults are depicted in Fig. 3 given below.

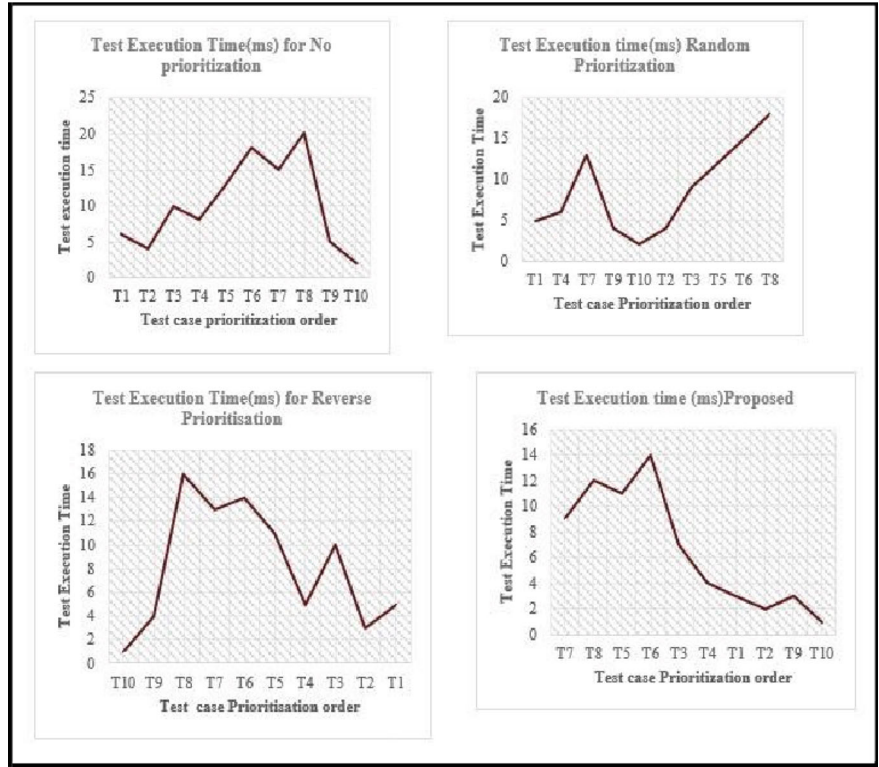


Fig. 2 Results for test execution time

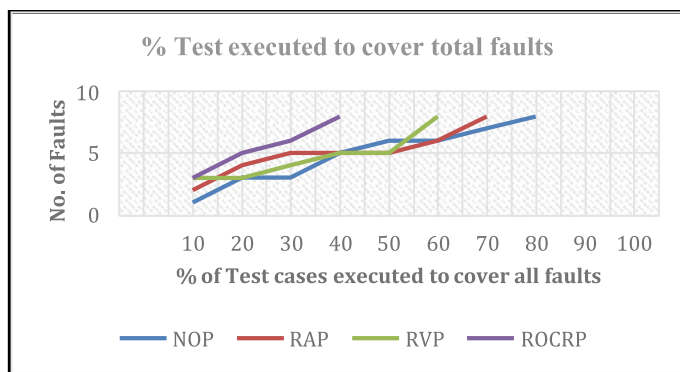


Fig. 3 Results for % of test cases executed to cover the faults

Table 5 Results to prove proposed work effectiveness

S. No	Test case prioritization technique	% Test suite to cover all faults	Total test case execution time (ms)
1	No prioritization	80%	101 ms
2	Random prioritization	70%	88 ms
3	Reverse prioritization	60%	82 ms
4	ROCR based (proposed)	40%	66 ms

The proposed system effectiveness is proved as given below in Table 5.

5 Conclusions and Future Scope

In software testing life cycle, if test case prioritization is done based on risk index (ROCR index) which is derived from the quality metrics, i.e., Software Resiliency, agility, elegance, business impact then the quality may be assured in terms of total execution time and % of test suite executed to cover all the faults which proves the correlation among risks, software testing life cycle, and quality assurance for the first RBT model and technique. One smart tool has been explored deeply to scan the evolutionary software's source code for identifying quality metrics, software health, and security vulnerabilities. The tool has been used to implement the cloud readiness and benchmarking of evolutionary software. The work has implemented test case prioritization based on the ranking of component risks and quality control metrics by automatic source code analysis for evolutionary software to correlate risk, quality assurance, and testing life cycle. The strong correlation among software testing life cycle, quality assurance, and risks has been established. This research work will prove

beneficial to society in terms how the emergent pervasive or artificial intelligent software applications requires risk assessment before testing them to prevent significant losses of things and people. In the future, the risk-based testing may be implemented for evolutionary environments like edge and cognitive computing where risks are more dynamic and unpredictable in nature.

References

1. Wanderley M, Menezes J Jr, Gusmaoa C, Limaa F (2015) Proposal of risk management metrics for multiple project software development. *Int Conf Enterp Inform Syst Publ Proc Comput Sci* 64:1001–1009
2. Tamburri DA, Palomba F, Kazman R (2020) Success and failure in software engineering: a follow up systematic literature review, *IEEE Trans Eng Manag*, pp 1–20 <https://doi.org/10.1109/TEM.2020.2976642>
3. Collier GD (1992) Engineers and risk issues, the engineering council code of practice, *IEEE XPLORE*, ieeexplore.ieee.org/document/168508/, pp 5/1–5/5
4. Oslon DL, Wu D (2020) Enterprise risk management in projects, enterprise risk management models, Springer texts in business and economics, pp 1–15
5. Khan S (2009) An approach to facilitate software risk identification, 2nd International conference on computer, control and communication, Karachi, pp 1–5
6. Weber KW, Jahne S, Krcmar H (2008) Relational risk mitigation, *Proceedings of the 41st Hawaii international conference on system sciences*, pp 1–10
7. Xiao-qing B, Qing-Xiang Z (2010) Notice of retraction: the identification method of the risk factors of knowledge sharing based on the evidence theory, 2010 IEEE International conference on advanced management science (ICAMS 2010), pp 630–634
8. Kang H (2010) The study on risk factors associated with information system project performance from knowledge resource's view, 2010 International conference on networking and digital society, Wenzhou, pp 444–446
9. Li X, Zhang M (2009) Research on the high-tech project risk pre-warning index optimization, 5th International conference on wireless communications, networking and mobile computing, Beijing, pp 1–4
10. Software testing life cycle, <http://tryqa.com/what-is-software-testing-life-cycle-stlc/>, Last Accessed Sept 2020
11. Erdogan G (2014) Approaches for the combined use of risk analysis and testing: a systematic literature review. *Int J Softw Tools Technol Transf* 16:627–642
12. Matheu-García SN (2019) Risk-based automated assessment and testing for the cybersecurity certification and labeling of IoT devices. *Comput Stand Interfaces* 62:64–83
13. Schechner B (2008) Getting started with software testing. pp 1–9
14. Kloos J, Hussain T, Eschbach R (2011) Risk-based testing of safety-critical embedded systems driven by fault tree analysis, In: 2011 IEEE Fourth international conference on software testing, Verification and Validation Workshops, pp 26–33
15. Yoon H, Choi B (2011) A test case prioritization based on degree of risk exposure and its empirical study. *Int J Softw Eng Knowl Eng* 21(02):191–209
16. Zech P (2011) Risk-based security testing in cloud computing environments, In: 2011 Fourth IEEE International conference on software testing, verification and validation, pp 411–414
17. Bai X, Kenett RS, Yu W (2012) Risk assessment and adaptive group testing of semantic web services. *Int J Softw Eng Knowl Eng* 22(05):595–620
18. Felderer M, Haisjackl C, Breu R, Motz J (2012) Integrating manual and automatic risk assessment for risk-based testing, In: International conference on software quality, pp 159–180

19. Felderer M, Ramler R (2013) Experiences and challenges of introducing risk-based testing in an industrial project, In: International conference on software quality, pp 10–29
20. Ray M, Mohapatra DP (2013) Risk analysis: a guiding force in the improvement of testing. *IET Softw* 7(1):29–46
21. Ramler R, Felderer M (2016) Requirements for integrating defect prediction and risk-based testing, In: 2016 42th Euromicro conference on software engineering and advanced applications (SEAA), pp 359–362
22. Großmann J, Felderer M, Viehmann J, Schieferdecker I (2019) A taxonomy to assess and tailor risk-based testing in recent testing standards. *IEEE Softw* 37(1):40–49
23. Ramler R, Felderer M, Leitner M (2016) A lightweight approach for estimating probability in risk-based software testing, In: International workshop on risk assessment and risk-driven testing, pp 115–128
24. GitHub—lucab85_PowerMeter-Reader_ Read energy data from Modbus power meter Schneider electric iEM3255—proof of concept, [Online]. Available: <https://github.com/lucab85/PowerMeter-Reader>
25. “GitHub—opencb_hpg-bigdata_ This repository implements converters and tools for working with NGS data in HPC or Hadoop cluster”, [Online]. Available: <https://github.com/opencb/hpg-bigdata>
26. GitHub—vitalets_awesome-smart-tv_ A curated list of awesome resources for building Smart TV apps, [Online]. Available: <https://github.com/vitalets/awesome-smart-tv>
27. GitHub-OpeNebula, [Online]. Available: <https://github.com/OpenNebula/one>
28. GitHub-pybrain_pybrain, [Online]. Available: <https://github.com/pybrain/pybrain>
29. CAST Highlight—rapid application portfolio analysis, [Online]. Available: <https://www.castsoftware.com/products/highlight>
30. CAST-highlight-indicators-methodology, [Online]. Available: <https://doc.casthighlight.com/methodology-analytic-upgrade/>
31. Qase test management tool, [Online]. Available: <https://qase.io/product>
32. Software testing magazine [Online, 2024]. Available: <https://www.softwaretestingmagazine.com/knowledge/optimizing-risk-based-testing-with-intelligent-automation/>
33. Khalil I, Imran G, Afsheen T, Amer N, Asif J, Naveed A, Sheeraz A (2024) Enhancing security testing through evolutionary techniques: a novel model, 6, 375–393
34. Leite GMC, Marcelino CG, Pedreira CE, Jiménez-Fernández S, Salcedo-Sanz S (2023) Evaluating the risk of uncertainty in smart grids with electric vehicles using an evolutionary swarm-intelligent algorithm. *J Clean Prod* 401:136775